

# Examen Final de GRAU-IA

(18 de junio de 2025)

**Duración: 2 horas 30 minutos**

1. (5 puntos) Debido al aumento de desastres naturales producidos por el cambio climático (las grandes inundaciones de la DANA en Valencia, el tifón Chido en las islas Mayotte, el huracán Helene en Estados Unidos) y otros por causas naturales (el terremoto de Vanuatu, el devastador tsunami del Océano Índico en 2004), la Oficina Para la Coordinación de Asuntos Humanitarios de la UNESCO ha decidido crear un SBC que ayude a tomar decisiones sobre el tipo de ayuda a enviar dependiendo de cada caso. El sistema se limitará a la resolución puntual de la respuesta rápida a emergencias súbitas ocurridas por desastres naturales (y no para desastres de duración prolongada como los incendios en Australia). Se quiere construir este SBC de manera que, en base al tipo de catástrofe, su magnitud y las características del lugar decida el tipo de ayuda que haga falta y haga una estimación del número de personas y del número de unidades de material de cada tipo que hay que enviar.

Para poder decidir el tipo de ayuda que se necesita enviar en cada caso, el sistema ha de recibir información sobre el desastre natural. Cada desastre sobre el que se ha de actuar recibe un nombre (por ejemplo "DANA Valencia" o "huracán Helene") y la fecha en que ocurrió el desastre. También se guarda el tipo de desastre (huracán/tifón, terremoto, inundación, erupción, desprendimiento de tierra...) y la magnitud del desastre, donde cada desastre tiene su propia escala de magnitudes: velocidad del viento (en km/h) para huracanes, grados en la Escala de Richter para terremotos, nivel del agua (en metros) y extensión inundada (en m<sup>2</sup>) para las inundaciones, radio de afectación (en km) y volumen de lava expulsada (en m<sup>3</sup>) para las erupciones, metros cúbicos de tierra para los desprendimientos, y así para el resto de tipos de desastre. También se necesita saber el número estimado de personas afectadas por el desastre que necesiten ayuda y el número estimado de heridos.

El sistema tendrá además información sobre el lugar del desastre (que se clasifica en isla, costa continental, interior continental). De los lugares se ha de guardar el nombre del país, el nombre del lugar (si el desastre es del tamaño de todo el país, el nombre del lugar coincidirá con el nombre del país), la latitud (en grados, un número con decimales) y la longitud (en grados, con decimales). También se necesita saber las características orográficas relevantes (valle, montaña, altiplano, desierto, estepa, río, lago, mar, oasis, glaciar) y para cada característica su nombre geográfico. En el caso de las montañas se ha de guardar también la altura máxima sobre el nivel del mar (en metros). En el caso de valles, altiplanos, desiertos y estepas se ha de guardar su extensión (en km<sup>2</sup>). Para las características orográficas que son fuentes de agua natural (río, lago, mar, oasis, glaciar, ...) se ha de guardar su nombre, la distancia (en km) a la población más cercana, si es navegable o no, y si el agua es potable, no potable pero potabilizable o no potabilizable. Hay que saber también el tipo de clima de la zona (tropical húmedo, desértico, seco, ártico...), la temperatura mínima y máxima del lugar según la previsión meteorológica para los próximos 7 días y en el caso de clima tropical saber también la humedad relativa esperada.

Hay que tener información de los servicios públicos que aun funcionan en el lugar (luz, agua canalizada, alcantarillado, gas, radio, telefonía), y para cada uno se ha de tener la estimación de su funcionamiento (en porcentaje, donde 100% significa servicio funcionando a la perfección) y en el caso del agua canalizada se ha de saber su grado de potabilidad (en porcentaje) después del desastre ya que a veces la calidad del agua canalizada empeora por roturas de las tuberías o por contaminación de las fuentes. También hace falta saber el estado de las vías de comunicación hasta el área del desastre (carreteras, vías de tren, puertos y aeropuertos) y para cada vía de comunicación se ha de guardar el tipo de vía y su estado actual (en porcentaje). Además se necesita saber si existen reservas de alimentos y medicinas en el lugar, pero no se guardará el detalle de cada alimento o medicina, solo una estimación de cuantos días durarán esas reservas. Y finalmente hace falta saber si en la zona del desastre existe algún conflicto militar o terrorista que pueda dificultar el suministro, y en ambos casos se ha de saber el número de milicianos o terroristas (respectivamente) y el nivel de peligrosidad asociado al conflicto (de 1 a 10).

Con toda esa información el sistema ha de poder identificar el volumen de afectados (pocos si son menos de 10000 personas, muchos en caso contrario); el volumen de heridos (pocos si son menos de 300, muchos si está entre 300 y 750, masacre si es mayor de 750); la disponibilidad de agua potable canalizada (si el servicio de agua canalizada tiene un grado de funcionamiento por debajo del 60% o un grado de potabilización por debajo del 80% entonces consideramos que no hay agua potable); la disponibilidad de una fuente de agua potable o potabilizable cerca (cerca si está a 20 km o menos de la población más cercana y su calidad

es potable o potabilizable, lejos si está a más 20 km de la población más cercana y su calidad es potable o potabilizable, ninguna en caso de fuentes no potabilizables); el nivel de destrucción (elevado en caso de terremotos por encima de los 6,5 grados Richter, huracanes por encima de los 250 km/h, erupciones con más de 5000 m<sup>3</sup> de lava expulsada o desprendimientos de tierra de más de 10000 m<sup>3</sup>, y leve en caso contrario ); el estado de las vías de comunicación (totalmente funcional si su estado es superior al 70%, parcialmente funcional si su estado está entre el 20% y el 70%, e inoperativa si está por debajo del 20%); el estado del servicio eléctrico (afectado si el grado de funcionamiento es por debajo del 60%, no afectado en caso contrario); el estado del servicio de telefonía (afectado si el grado de funcionamiento es por debajo del 50%, no afectado en caso contrario) y finalmente si es una zona peligrosa para el personal de ayuda (mucho si el grado de peligrosidad del conflicto es por encima del 6, poco si está entre el 6 y el 2, nada si está por debajo del 2).

La ayuda puede ser en material o en personas. El material de ayuda se mide en número de unidades (cada material tiene asociado un peso por unidad, en kg) y puede ser de muchos tipos: consumibles (packs de alimentos, packs de medicinas, cisternas de agua y garrafas de combustible), ropa, material de campaña (tiendas de campaña y hospitales de campaña, en ambos casos se guarda la capacidad en número de personas), material de servicios (generadores de electricidad, potabilizadoras y equipos de comunicaciones), maquinaria pesada (grúas y excavadoras, en ambos casos se guarda el peso máximo en toneladas que pueden mover) y vehículos (aviones, barcos, camiones y helicópteros, en todos los casos se guarda el volumen de carga -en m<sup>3</sup>- y el peso máximo de carga -en toneladas-). Además la ayuda también puede incluir personal humano: médicos, psicólogos, traductores, bomberos, ingenieros civiles, expertos en logística, pilotos (que pueden ser pilotos de aviones, barcos, camiones o helicópteros), operadores de maquinaria pesada (que pueden estar especializados en gruas o en excavadoras), policías y efectivos militares (para proteger al resto del personal humano y/o en zonas de guerra). Para todos estos casos no hace falta guardar información de personas individuales, sino el número de médicos, psicólogos, traductores... que hará falta movilizar en la zona.

Los expertos en logística de desastres naturales nos han dado ejemplos sobre como toman algunas de las decisiones a la hora de decidir la ayuda a enviar:

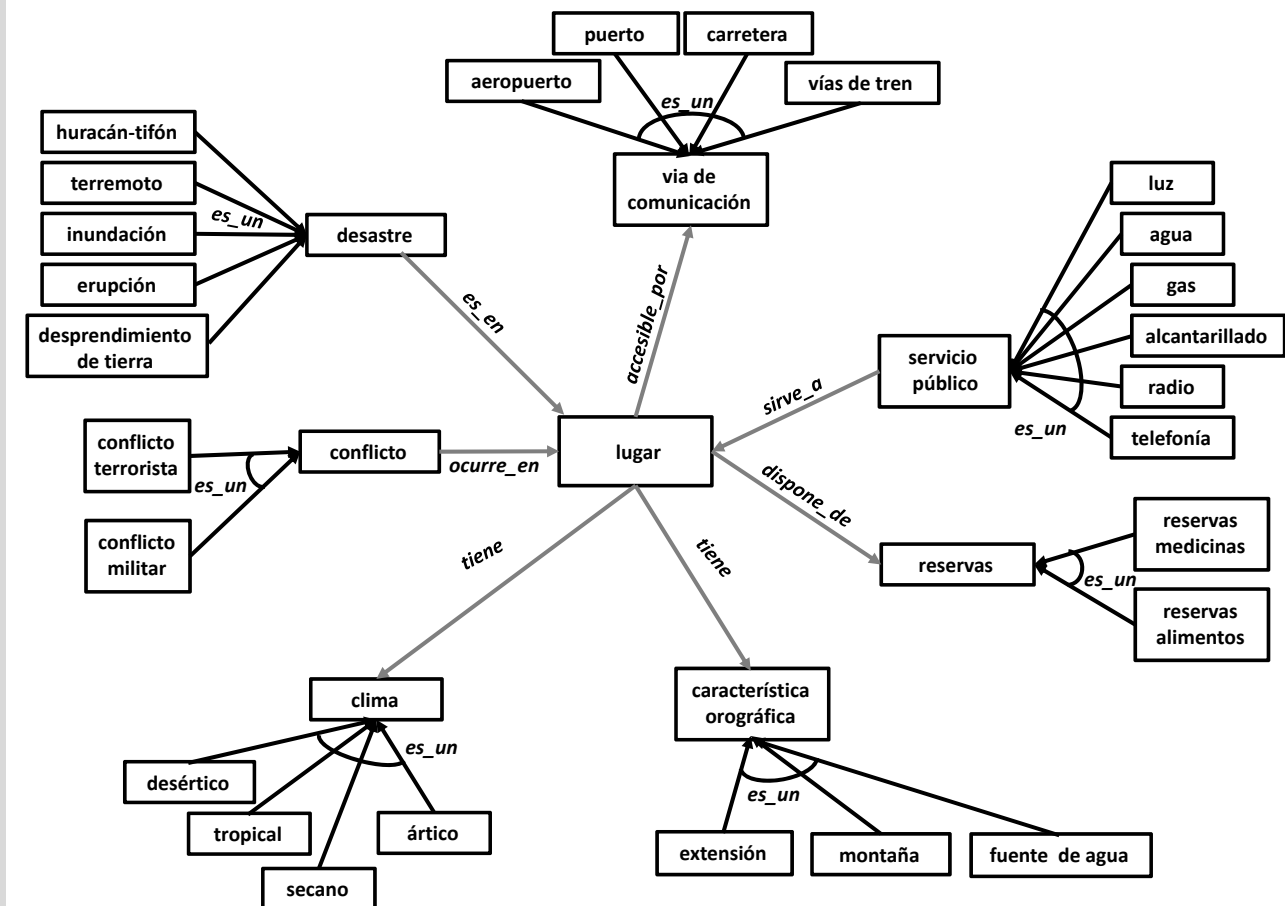
- si el volumen de afectados es elevado se ha de incluir muchos alimentos y muchos psicólogos; si hay un volumen de heridos elevado o masacre se han de incluir muchos hospitales de campaña, muchos médicos y muchas medicinas;
- si el nivel de destrucción es elevado o el estado de las vías de comunicación es impracticable se han de enviar muchos ingenieros civiles y mucha maquinaria pesada; si el nivel de destrucción es leve y las vías de comunicación son funcionales (parcialmente o totalmente) se envían pocos ingenieros civiles y maquinaria;
- si hay un volumen de afectados elevado, hay fuentes de agua utilizable para el consumo cerca del lugar y la calidad de ese agua es potable o potabilizable se enviarán muchas potabilizadoras; si hay un volumen de afectados elevado y no hay fuentes cerca o el agua no es potabilizable se mandarán muchas cisternas de agua potable;
- si el volumen de afectados es muchos y el servicio eléctrico del lugar está afectado hay que llevar muchos generadores de electricidad; si el volumen de afectados es pocos y el servicio eléctrico del lugar está afectado hay que llevar pocos generadores de electricidad; si el volumen de afectados es muchos y el servicio de telefonía está afectado hay que llevar muchos equipos de comunicaciones; si el volumen de afectados es pocos y el servicio de telefonía está afectado hay que llevar pocos equipos de comunicaciones;
- si la zona es muy peligrosa se han de movilizar muchos militares; si hay muchos médicos o muchos psicólogos o muchos ingenieros civiles o muchos militares harán falta muchos traductores;
- si hay mucho material o mucho personal de ayuda que mover se usarán muchos vehículos; si hay muchos vehículos harán falta muchos pilotos y mucho combustible;
- si hacen falta muchos médicos y muchos hospitales de campaña, enviar un hospital de campaña por cada 200 heridos y un médico por cada 60 heridos;
- si hacen falta pocos médicos y pocos hospitales de campaña, enviar un hospital de campaña por cada 100 heridos y un médico por cada 30 heridos;
- si hay un aeropuerto con un estado de funcionamiento superior al 60% todo el personal y parte del material de primera necesidad (medicinas, alimentos, tiendas de campaña, hospitales de campaña) se enviará en avión;

- si hay un puerto con un estado de funcionamiento superior al 40% el material de servicios, la maquinaria pesada y los vehículos se enviarán por barco;
  - si el estado de las carreteras es superior al 40% la ayuda se distribuirá en camión; si las carreteras están demasiado dañadas la ayuda se distribuirá en helicópteros
  - el número exacto de vehículos de cada tipo se calculará a partir del numero de personas, y del peso total de las unidades de material a mover;
  - el número exacto de pilotos de cada tipo se calculará a partir del número de vehículos de cada tipo, teniendo en cuenta que en algunos vehículos (aviones, helicópteros) se requiere más de un piloto.
  - se enviará un experto en logística por cada 90 personas que se envían;
  - si hacen falta muchos o pocos militares se enviará un militar por cada 3 milicianos en la zona, y 10 militares por cada terrorista.
- a) (2,5 puntos) Diseña la ontología del dominio descrito, incluyendo todos los conceptos que aparecen en la descripción. **Lista explícitamente** todos los atributos para categorías de *lugar*, *desastre*, *personal* y *material* (para cada atributo da un nombre y su tipo). Lista que conceptos forman parte de los datos de entrada del problema y que conceptos forman parte de la solución. (Nota: tened en cuenta que la ontología puede necesitar modificaciones para adaptarla al apartado siguiente).

En este problema la ontología ha de incorporar, como mínimo, todos los conceptos que forman parte de la entrada del problema, así como conceptos que formen parte de la solución.

Los conceptos que forman parte de la entrada del problema son todos referidos a características del desastre sobre el que se ha de actuar:

- la clase del desastre
- quien se ha visto afectado
- las características del lugar donde ha ocurrido
- las reservas disponibles de alimentos y medicinas
- los servicios públicos que aún están en funcionamiento
- las vías de comunicación disponibles



Existen varios conceptos que comparten atributos y/o relaciones, y que esto ha sido algo que se ha tenido muy en cuenta a la hora de crear super-clases con las características comunes, o para decidir entre crear subclases o solo añadir un atributo **tipo**. La regla general es que hemos creado subclases cuando una o varias tienen atributos diferentes y/o relaciones diferentes. Ese es el caso de las subclases de **desastre** (cada desastre tiene magnitudes diferentes o más de una magnitud), **característica orográfica** (dos subclases tienen atributos diferentes), **clima** (una subclase tiene atributos diferentes), **conflicto** (ambas subclases tienen atributos diferentes) o de **servicio público** (una subclase tiene atributos diferentes). Habríamos podido añadir las subclases **valle**, **Altiplano**, **estepa** y **desierto**, pero como todas comparten el mismo atributo, se han fusionado en una clase **extensión** que distingue entre ellas con un atributo de tipo. En vez de distinguir subclases de **fuentes de agua**, como todas tienen los mismos atributos se ha creado un atributo de tipo.

Cabe remarcar que en el caso de las clases **reservas** y **vía de comunicación** se ha decidido desplegar un nivel de subclases (a pesar de que no tienen ni atributos ni relaciones diferentes) siguiendo el heurístico del diseño de ontologías que recomienda que ramas hermanas de la taxonomía tengan un nivel de granularidad similar. Y que en el caso de la clase **desastre** no se ha dibujado un arco entre las flechas de las subclases ya que no presentan una clasificación completa (en el enunciado, el listado de tipos de desastre sugiere que pueden existir otros tipos de desastre que no son los que nosotros hemos modelado).

Atributos:

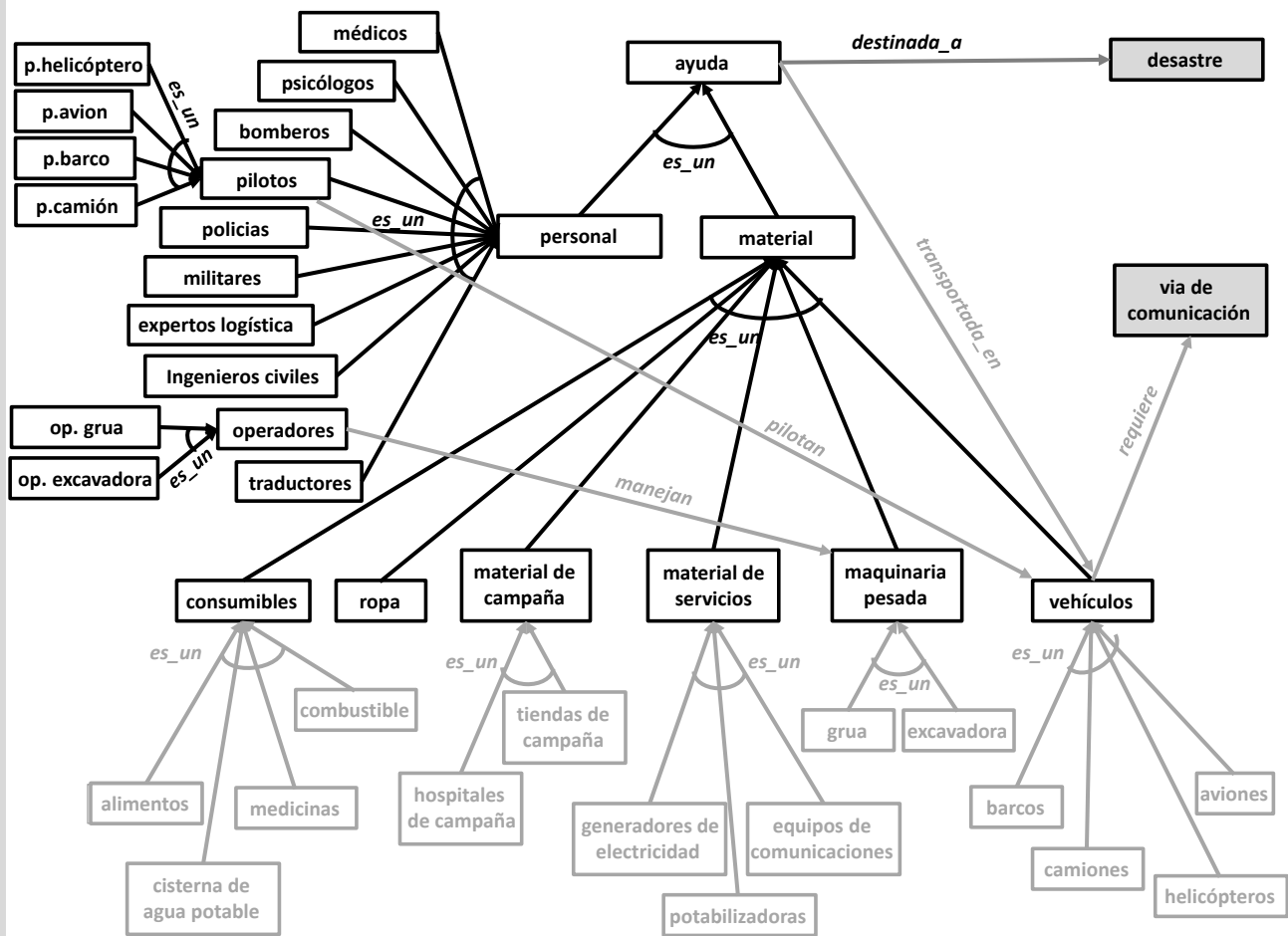
- **desastre**: nombre (string), fecha (fecha), num\_afectados (entero positivo), num\_heridos (entero positivo), *vol\_afectados* (enumeración: {pocos, muchos}), *vol\_heridos* (enumeración: {pocos, muchos, masacre}), *nivel\_daños* (enumeración: {leve, elevado})
- **huracán-tifón**: velocidad\_viento (km/h)
- **terremoto**: grados\_richter (real positivo)

- **inundación:** nivel\_agua (metros), m2\_inundados (metros cuadrados)
- **erupción:** radio\_de\_afectación (km), volumen\_lava (metros cúbicos)
- **desprendimiento de tierra:** m3\_tierra (metros cúbicos)
- **lugar:** nombre\_pais (string), nombre\_lugar (string), latitud (grados), longitud (grados), tipo (enumeración: {isla, costa\_continental, interior\_continental }, *fuentes\_disponibles* (enumeración: {cerca, lejos, ninguna})), *estado\_servicio\_agua* (enumeración: {disponible, no\_disponible}), *estado\_servicio\_eléctrico* (enumeración: {afectado, no\_afectado}), *estado\_servicio\_telefónico* (enumeración: {afectado, no\_afectado}), *estado\_vías* (enumeración: {totalmente\_funcional, parcialmente\_funcional, inoperativa}), *zona\_peligrosa* (enumeración: {mucho, poco, nada})
- **característica orográfica:** nombre (string);
- **montaña:** altura\_máxima (metros)
- **extensión:** area (km cuadrados), tipo (enumeración: {valle, estepa, altiplano, desierto })
- **fuentes de agua:** distancia (km), navegable (booleano), potable (enumeración: {si, potabilizable, no}), tipo (enumeración: {rio, lago, oasis, mar, glaciar })
- **clima:** temp\_max (grados), temp\_min (grados)
- **tropical:** humedad ( %),
- **servicio público:** grado\_funcionamiento ( %)
- **agua:** potable ( %)
- **via de comunicación:** estado\_actual ( %)
- **reservas:** estimación\_días (entero positivo)
- **conflicto:** peligrosidad (entero positivo)
- **conflicto\_militar:** num\_milicias (entero positivo)
- **conflicto\_terrorista:** num\_terroristas (entero positivo)

Como se puede ver, hemos decidido también representar las características abstractas del problema (son los que tienen el nombre en *Cursiva*). De esta manera todos los conceptos que aparecerán en las reglas están soportados por la ontología.

En el caso de los atributos solo cabe destacar que, aunque todos los desastres tienen algún tipo de magnitud, no es buena idea poner la magnitud como atributo de **desastre**, ya que el enunciado nos dice que cada tipo de desastre tiene diferentes formas de medir su magnitud e incluso tenemos dos tipos de desastre que tienen más de una magnitud. Por ello en la lista de atributos se puede ver que se han puesto atributos distintos para los diferentes tipos de desastre, que se ajustan mejor a cada uno de ellos.

La ontología asociada a la solución esta compuesta por los diferentes tipos de ayuda y las cantidades a enviar.



#### Atributos:

- **personal**: num\_personas (entero positivo), *REC* (enumeración: {muchas, pocas, ninguna})
- **material**: num\_unidades (entero positivo), peso\_por\_unidad (kg), *REC* (enumeración: {mucho, poco, nada})
- **material de campaña**: capacidad (entero positivo)
- **maquinaria pesada**: peso\_max (toneladas)
- **vehículos**: vol\_carga (metros cúbicos), carga\_max (toneladas)

Como se puede ver, hemos decidido también representar las características abstractas de la solución abstracta (son los que tienen el nombre en *CURSIVA*). De esta manera todos los conceptos que aparecerán en las reglas están soportados por la ontología.

Esta parte de la ontología está dominada por relaciones taxonómicas. También se ha representado como se conecta esta parte de la ontología con la anterior, a través del una relación *destinada\_a* (de *ayuda* hacia el concepto *desastre*) y otra relación *requiere* (de *vehículo* hacia *vía de comunicación*). Se ha optado por tener el concepto *ayuda* con dos subclases (*material* y *personal* de *ayuda*) porque tienen atributos diferentes. También se han definido subclases de *material* (algunas tiene atributos propios). Se ha añadido un nivel más de subclases de *material* (en gris en el diagrama) ya que se van a usar esos conceptos directamente en el apartado b, pero también sería correcto no crear subclases en este caso y crear atributos de tipo en *consumibles*, *material de campaña*, *material de servicios*, *maquinaria pesada* y *vehículos*.

En el caso de *personal*, aunque a priori puede parecer que las subclases no tiene ni atributos ni relaciones distintas (todas ellas tienen un número de personas y una recomendación, y dado que no distinguimos ni pilotos ni operadores de forma individual, tampoco hacer falta indicar cada piloto que vehículo maneja, ni cada operador que maquinaria maneja), se ha decidido desplegar dos niveles de subclases de forma que, para toda subclase, tenemos recomendaciones y números de personas necesarias. La alternativa en este caso sería no desplegar las subclases de persona y

tener un atributo **tipo de persona** enumerando todos los tipos de personal, pero en ese caso sería muy importante incluir en la enumeración los subtipos de pilotos y de operadores para poder tener recomendaciones y contadores separados de número de personas para cada uno de los subtipos. Se han puesto en gris las relaciones entre pilotos y vehículos y la relación entre operadores y maquinarias porque en este problema, al no tener identificados pilotos operadores individuales ni vehículos o maquinaria particulares, no es imprescindible modelar las relaciones entre las personas y lo que manejan (pero tampoco es erróneo añadir estas relaciones). Se podría incluso relacionar directamente las subclases de **pilotos** con las subclases de **vehículos** y las subclases de **operadores** con las subclases de **maquinaria pesada**, pero se ha decidido no complicar aun más el diagrama de la ontología innecesariamente.

Es importante destacar también que en este caso se han puesto los conceptos de la ontología en plural, y no en singular, ya que en este caso no pretenden representar unidades concretas de material o un individuo concreto dentro del personal.

- b) (2,5 puntos) El problema descrito es un problema de análisis. Explica cómo lo resolverías usando clasificación heurística, usando los conceptos de la ontología desarrollada en el apartado anterior. Da al menos 4 ejemplos de reglas (suficientemente variadas, utilizando diferentes conceptos) para cada una de las fases de esta metodología.

Para resolverlo mediante clasificación heurística debemos tener un conjunto enumerable de soluciones candidatas. Por ello plantearemos la solución partiendo de la hipótesis que existe:

- un conjunto enumerable a priori de tipos de catástrofe (Terremoto menor, terremoto mayor, huracán continental medio, huracán en isla leve, ...) o de características abstractas que nos permiten describir estos tipos de catástrofe.
- un conjunto enumerable a priori de soluciones tipo a estas catástrofes, o un conjunto enumerable de elementos de la solución abstracta.

En este caso se ha optado por tener un conjunto finito de características abstractas, para describir el problema, y un conjunto finito de elementos que describen la solución abstracta. A partir de este planteamiento lo que nos queda es describir las fases y elementos necesarios para aplicar la clasificación heurística al problema.

El primer elemento son los problemas concretos que hay que tratar. En este dominio los problemas concretos están definidos por los datos concretos del desastre sobre el que se ha de actuar (clase de desastre, lugar, número de afectados...) tal y como se reciben de entrada.

El segundo elemento son los problemas abstractos, que describen las características abstractas del desastre (destrucción elevada, pocos heridos, muchos heridos, sin fuentes de agua potable, sin fuentes de agua potabilizable, vías de comunicación afectadas, zona peligrosa...).

Para conectar los problemas concretos con los abstractos necesitamos definir las reglas de abstracción de datos, por ejemplo:

- si `desastre.num_heridos < 500` entonces `desastre.vol_heridos=pocos`;
- si `desastre.num_heridos >= 750` entonces `desastre.vol_heridos=masacre`;
- si `desastre.num_afectados > 10000` entonces `desastre.vol_afectados=muchos`;
- si `terremoto.grados_richter > 6.5` o `huracán-tifón.velocidad_viento > 250` o `erupción.volumen_lava > 5000` o `desprendimiento_de_tierra.m3_tierra > 10000` entonces `desastre.nivel_daños=elevado`;
- si `agua.potable >= 80 %` y `agua.grado_funcionamiento >= 60 %` entonces `lugar.estado_servicio_agua=disponible`;
- si `agua.potable < 80 %` o `agua.grado_funcionamiento < 60 %` entonces `lugar.estado_servicio_agua=no_disponible`;
- si existe (`conflicto.peligrosidad > 6`) entonces `lugar.zona_peligrosa=mucho`;
- si media (`vía_de_comunicación.estado_actual > 70 %`) entonces `lugar.estado_vías=totalmente_funcional`;

El tercer elemento son las soluciones abstractas. En este caso podríamos tener arquetipos de soluciones completas (si las tuviéramos muy bien definidas por algún tipo de protocolo de actuación) o simplemente identificar de forma abstracta que elementos necesitamos para solucionar el desastre.

Para ligar los problemas abstractos con las soluciones abstractas necesitaremos reglas de asociación heurística, como por ejemplo:

- si desastre.vol\_afectados=muchos entonces pack\_alimentos.REC=mucho  
y psicólogos.REC=muchos;
- si desastre.vol\_heridos=(muchos o masacre) entonces hospitales\_de\_campaña.REC=mucho;  
y pack\_medicinas.REC=mucho y médicos.REC=muchos
- si desastre.vol\_heridos=pocos entonces hospitales\_de\_campaña.REC=poco  
y médicos.REC=pocos y pack\_medicinas.REC=poco;
- si lugar.estado\_servicio\_agua=no\_disponible y lugar.fuente\_disponible=cerca  
y desastre.vol\_afectados=muchos entonces potabilizadoras.REC=mucho;
- si lugar.estado\_servicio\_agua=no\_disponible y lugar.fuente\_disponible=lejos  
y desastre.vol\_afectados=muchos entonces cisternas\_de\_agua\_potable.REC=mucha;
- si lugar.estado\_servicio\_eléctrico=afectado y desastre.vol\_afectados=muchos  
entonces generadores\_de\_electricidad.REC=mucho;
- si lugar.estado\_servicio\_telefónico=afectado y desastre.vol\_afectados=pocos  
entonces equipos\_de\_comunicación.REC=poco;
- si lugar.zona\_peligrosa=mucho entonces militares.REC=muchos;
- si médicos.REC=muchos o psicólogos.REC=muchos o ingenieros\_civiles.REC=muchos  
o militares.REC=muchos entonces traductores.REC=muchos;

El cuarto elemento son las soluciones concretas. En este caso son soluciones completas en las que se tiene el número de unidades y efectivos necesarios.

Para llegar a las soluciones concretas se han de refinar las soluciones abstractas, usando información concreta del desastre (su magnitud, el número de afectados...). Algunos ejemplos de como refinar la solución son los siguientes:

- si médicos.REC=muchos y hospitales\_de\_campaña.REC=mucho entonces  
( hospitales\_de\_campaña.num\_unidades= redondear(desastre.num\_heridos/200);  
médicos.num\_personas = redondear(desastre.num\_heridos/60); )
- si médicos.REC=pocos y hospitales\_de\_campaña.REC=poco entonces  
( hospitales\_de\_campaña.num\_unidades= redondear(desastre.num\_heridos/100);  
médicos.num\_personas = redondear(desastre.num\_heridos/30); )
- si cierto entonces  
( experto\_en\_logística.num\_personas = Cardinalidad(personal.destinado\_a(desastre))/90; )
- si existe(Aeropuerto.estado\_actual > 60%) entonces  
personal.transportado\_en(avion) y pack\_medicinas.transportado\_en(avion)  
y pack\_alimentos.transportado\_en(avion) y tiendas\_de\_campaña.transportado\_en(avion)  
y hospitales\_de\_campaña.transportado\_en(avion);
- si existe(Puertos.estado\_actual > 40%) entonces  
material\_de\_servicios.transportado\_en(barco) y maquinaria\_pesada.transportado\_en(barco) y  
camiones.transportado\_en(barco) y helicópteros.transportado\_en(barco);
- si media(Carretera.estado\_actual > 40%) entonces  
personal.transportado\_en(camión) y material.transportado\_en(camión);
- si media(Carretera.estado\_actual <= 40%) entonces  
personal.transportado\_en(helicóptero) y material.transportado\_en(helicóptero);
- si existe(ayuda.transportado\_en(avión) entonces  
( vol = estima\_volumen\_total(material.transportado\_en(avion));  
vol = vol + estima\_espacio\_necesario(personal.transportado\_en(avion))  
peso = sumatorio((material.transportado\_en(avion)).num\_unidades  
\*(material.transportado\_en(avion)).peso\_unidad);  
peso = peso + 75\*(sumatorio((personal.transportado\_en(avion)).num\_personas));  
avión.num\_unidades = Redondear(Max(vol/avión.vol\_carga, peso/avión.peso\_max)); )
- si militares.REC=(muchos o pocos) entonces  
( militares.num\_personas = sumatorio(conflicto\_militar.num\_milicias)/3  
+ 10\*sumatorio(conflicto\_terrorista.num\_terroristas); )



2. (5 puntos) En un intento de atraer más visitantes, la red de albergues *Future Hostel* está buscando nuevos servicios que ofrecer a sus clientes. Tras un acuerdo firmado con una start-up que construye robots humanoides, los albergues de la red ofrecerán un servicio de habitaciones robotizado por las noches, cuando no hay personal humano en el albergue. Los clientes podrán solicitar a través de una app algunos productos (mantas, almohadas, toallas, papel de baño, adaptadores de enchufe, snacks, refrescos, cerveza, agua mineral y agua con gas) que les serán llevados a su dormitorio. Además, los clientes podrán pedir también algunas preparaciones recién hechas (sandwiches, omelettes y zumo de naranja).

Un planificador recibe las peticiones, cada una de ellas incluye el cliente, el dormitorio compartido en el que duerme, y el producto o preparación que solicitan, y crea un plan para servir las peticiones con los robots disponibles. En el caso de los productos, estos se encuentran almacenados en diferentes lugares del albergue (la cocina, almacenes), y los robots asistentes se usarán para llevar esos productos desde el lugar de almacenaje hasta el dormitorio del cliente. Para transportar los productos estos robots asistentes tienen contenedores especiales (normalmente 2) instalados en sus cuerpos. Un robot asistente solo puede llevar un producto o preparación por contenedor, y no puede coger más productos/preparaciones si tiene todos sus contenedores llenos. Para los productos almacenados consideramos que hay más que suficientes para satisfacer todas las demandas de una noche, y por lo tanto un robot que vaya a buscar un producto a su lugar de almacenamiento siempre encontrará uno disponible. En el caso de preparaciones recién hechas los robots asistentes deben esperar a que la preparación sea realizada por un tipo especial de robot, el robot cocinero. Los robots cocineros solo son capaces de realizar las preparaciones, y solo las concinarán bajo demanda, es decir, que solo cocinarán una preparación cuando aparece una petición del cliente. Los robots asistentes solo irán a buscar las preparaciones a la cocina cuando estén listas, y será entonces cuando las lleven en sus contenedores al dormitorio correspondiente.

La compañía quiere que el planificador modele todos los pasos de la solución: el robot cocinero cocina las preparaciones, el robot asistente va al lugar donde el producto/preparación está disponible, el robot asistente recoge el producto/preparación del lugar en el que se encuentra, el robot asistente va a un dormitorio, el robot asistente entrega el/los producto(s)/preparación(es) que corresponde(n) a ese dormitorio al cliente correcto.

- a) Describe el dominio (incluyendo predicados, acciones, etc...) usando PDDL. Da una explicación razonada de los elementos que has escogido. Ten en cuenta que el modelo del dominio ha de poderse extender a más o menos dormitorios, clientes, robots cocineros, robots asistentes (con más o menos contenedores).

Existen muchas posibles formas de modelar este dominio en PDDL (con más o menos predicados, con más o menos tipos, con más o menos operadores...) y por ello tomaremos decisiones que vayan encaminadas a crear un modelo que no contenga ineficiencias innecesarias.

Además de lo habitualmente comentado en las rúbricas de estos exámenes (no usar forall de forma innecesaria, minimizar operadores, etc.) listaremos errores bastante comunes junto con su penalización. La dificultad principal yace en que, si no se piensa bien el manejo de tener múltiples ítems repetidos, el programa puede quedar inutilizado (e.g. robots que no pueden coger dos sandwiches, preparar un sandwich implica tener infinitos, etc). Esto se trivializa si se trabaja con pedidos como veremos en el ejemplo inferior.

Errores comunes:

- (-0.25 por infracción) El programa no se puede extender a más de un almacén, más o menos de 2 contenedores por robot, más o menos robots, etc.
- (-0.5 a -1) Las preparaciones no se preparan y siempre están disponibles.
- (-0.25) No se distingue la entrega entre clientes (e.g. se entrega en una habitación, sin saber a quién).
- (-0.25) (tiene ?r ?item): hace que un robot no pueda llevar dos del mismo ítem a la vez. Si es un identificador de pedido, esto no causa problemas y no se penaliza.
- (-0.25) Preparar/cocinar 'deja' el ítem disponible en cualquier cocina (debería decir en qué cocina). No se penaliza si ya se ha penalizado por extensibilidad de número de cocinas.
- (-0.25) Preparar una vez permite coger la preparación infinitas veces.
- (-0.25) Alternativamente, preparar una vez solo permite coger la preparación una vez e impide volver a preparar ese ítem.
- (-0.25 a -0.5) Acciones repetidas innecesariamente (coger\_producto y coger\_preparación;

ir\_cocina, ir\_dormitorio e ir\_almacén...). Se ha perdonado si la lógica es suficientemente diferente, pero rara vez ha sido el caso.

- (-0.1) Se pueden cocinar más preparaciones de las pedidas (e.g. se piden 3 sandwiches pero el programa permite hacer 10). Tanto para preparaciones como para pedidos, esto ensancha el espacio de búsqueda de forma innecesaria haciendo la tarea ineficiente.
- (-0.1) Se pueden coger más productos de los pedidos.

Por contra, se han dado puntos extra por los siguientes éxitos, que no eran triviales:

- Manejar de forma correcta múltiples ítems, e.g. permitiendo que haya dos 'sandwiches' en la misma cocina para recoger a la vez, o que el robot los lleve.
- El robot asistente no puede esperar en la cocina a que algo esté preparado (explícitamente prohibido por el enunciado). Difícil de resolver de forma sucinta, se puede impedir con fluentes contando cuántos asistentes hay en cada cocina y solo permitir preparar si hay 0, mezclando la acción de ir y recoger en cocina (ir\_y\_recoger) además de tener la acción de ir, tener un forall en la precondition de preparar... Optamos por este último (a pesar de la ineficiencia) ya que la acción solo se puede correr tantas veces como el número de preparaciones dado el (preparado ?p).

Para nuestra solución propuesta, centramos el problema en 'pedidos' individuales, de forma que no se puede cocinar más preparaciones de las necesarias por pedidos, podemos llevar el mismo producto/preparación en el mismo robot, etc. Además, impedimos que se puedan coger más productos que pedidos haya que los necesiten. Dos posibles pegas a la solución propuesta son: el uso de forall en la precondition (se puede substituir por fluentes, pero hace el problema más ineficiente) y la distinción entre recoger un producto y una preparación, que justificamos porque recoger un producto implica 'crear' el pedido, mientras que recoger una preparación implica recoger un pedido ya creado por el cocinero. Usamos slots abstractos, teniendo tantos slots como máximo número de slots pueda tener un robot, pero se podrían tener multiplexados (la suma de slots por robot) y usar predicado (libre S) en lugar de (libre ?S ?R), como vimos en clase.

```
(define (domain albergue)
  (:requirements :adl :typing)
  (:types lugar item slot cliente - object
           cocina almacen dormitorio - lugar
           pedido robot elemento - item
           cocinero asistente - robot
           producto preparacion - elemento
  )

  (:predicates
    (pedido_por ?p - pedido ?cl - cliente ?d - dormitorio)
    (elemento_pedido ?p - pedido ?e - elemento)
    (compatible ?pr - preparacion ?co - cocina)
    ;; Por si hay cocinas que no pueden preparar ciertas comidas
    (preparado ?p - pedido)
    (recogido ?p - pedido)
    (entregado ?p - pedido)
    (esta_en ?x - item ?l - lugar)
    (libre ?r - asistente ?s - slot)
    (cargado_en ?p - pedido ?s - slot ?r - asistente)
  )

  (:action crear_preparacion
    :parameters (?rc - cocinero ?co - cocina ?pr - preparacion ?p - pedido)
    :precondition (and (not (preparado ?p)) (elemento_pedido ?p ?pr)
                       (compatible ?pr ?co) (esta_en ?rc ?co)
                       (forall (?a - asistente) (not (esta_en ?a ?co)))) )
    :effect (and (esta_en ?p ?co) (preparado ?p))
  )
)
```

```

(:action ir
  :parameters (?r - asistente ?orig ?dest - lugar)
  :precondition (esta_en ?r ?orig)
  :effect (and (not (esta_en ?r ?orig)) (esta_en ?r ?dest))
)
;; Implícitamente, el robot no puede ir a la cocina y esperar a preparaciones.

;; Distinguimos recoger preparaciones (el pedido está creado y en un sitio)
;; y productos (hay que 'hacer' el pedido y no se agotan existencias
;; del elemento)
(:action recoger_preparacion
  :parameters (?r - asistente ?l - cocina ?p - pedido
  :precondition (and (not (recogido ?p)) (esta_en ?r ?l) (esta_en ?p ?l)
    (libre ?r ?s))
  :effect (and (recogido ?p) (not (esta_en ?p ?l)) (not (libre ?r ?s))
    (cargado_en ?p ?s ?r))
)

(:action recoger_producto
  :parameters (?r - asistente ?l - lugar ?p - pedido ?pr - producto ?s - slot)
  ;; Puede ser en un almacen o en una cocina, por tanto 'lugar'
  :precondition (and (not (recogido ?p)) (elemento_pedido ?p ?pr) (esta_en ?r ?l)
    (esta_en ?pr ?l))
    (libre ?r ?s))
  :effect (and (recogido ?p) (not (libre ?r ?s)) (cargado_en ?p ?s ?r))
)

(:action entregar
  :parameters (?r - asistente ?d -dormitorio ?cl - cliente ?p - pedido ?s - slot)
  :precondition (and (not (entregado ?p)) (esta_en ?r ?d)
    (pedido_por ?p ?cl ?d)
    (cargado_en ?p ?s ?r) )
  :effect (and (entregado ?p) (libre ?r ?s) (not (cargado_en ?p ?s ?r))
    (esta_en ?p ?d))
)
)

```

- b) El jefe de Future Hostels nos ha proporcionado una tabla como ejemplo de las peticiones que el sistema recibe cada 30 minutos, con una serie de productos/preparaciones que se han de transportar desde dos almacenes (almacen1 para las mantas, almohadas y toallas, almacen2 para el papel de baño y los adaptadores de enchufe) o desde la única cocina en el hostel (snacks, refrescos, cerveza, agua mineral, agua con gas, sandwiches\*, omelettes\* y zumo de naranja\*), donde el \* indica que son productos frescos recién hechos. Cada uno de esos productos/preparaciones debe ser transportada a uno de los 6 dormitorios en el albergue (H101, H102, H103, H201, H202, H203) También nos dice que al inicio de ese intervalo de 30 minutos tenemos solo el robot cocinero *R1C1* en la cocina, y dos robots asistentes, *r2W1* and *r2W2* que esperan los pedidos desde *Almacen1* y *Almacen2* respectivamente. Se nos pide que el planificador genere un plan en el que los robots disponibles sirvan todos los pedidos, transportando cada producto/preparación al dormitorio correspondiente.

| cliente  | dormitorio | producto o preparación* |
|----------|------------|-------------------------|
| Virginia | H102       | manta                   |
| Sam      | H101       | almohada                |
| Rick     | H101       | sandwich*               |
| Catalina | H201       | toalla                  |
| Ana      | H201       | zumoS Naranja*          |
| Paul     | H103       | omelette*               |
| Agnes    | H203       | papelWC                 |
| Manuel   | H202       | sandwich*               |
| Virginia | H102       | adaptador               |
| Frank    | H202       | cerveza                 |

Describe este problema usando PDDL. Da una breve explicación de cómo modelas el problema.

En este caso el modelado del problema está totalmente marcado por el modelado del dominio del apartado anterior. El resultado es el siguiente:

```
(define (problem 1kitchen2lockers1cook2assistants6rooms10orders)
  (:domain albergue)
  (:objects cocinal - cocina
            almacen1 almacen2 - almacen
            H101 H102 H103 H201 H202 H203 - dormitorio
            R1C1 - cocinero
            r2W1 r2W2 - asistente
            S1 S2 - slot
            Virginia Sam Rick Catalina Ana Paul Agnes Manuel Frank - cliente
            manta almohada toalla papelWC adaptador - producto
            snack refresco cerveza aguaMineral aguaConGas - producto
            sandwich omelette zumoS Naranja - preparacion
            P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 - pedido
  )

  (:init
    (pedido_por P1 Virginia H102) ... (pedido_por P10 Frank H202)
    (elemento_pedido P1 manta) ... (elemento_pedido P10 cerveza)
    (compatible sandwich cocinal) (compatible zumoS Naranja cocinal) ...
    (esta_en R1C1 cocinal)
    (esta_en r2W1 almacen1) (esta_en r2W2 almacen2)
    (esta_en manta almacen1) (esta_en almohada almacen1) (esta_en toalla almacen1)
    (esta_en papelWC almacen2) (esta_en adaptador almacen2)
    (esta_en snack cocinal) (esta_en refresco cocinal) (esta_en cerveza cocinal)
    ;; sandwiches y otras preparaciones no esta_en ningún sitio
    (libre r2W1 S1) (libre r2W1 S2)
    (libre r2W2 S1) (libre r2W2 S2)
  )

  (:goal (forall (?p - pedido) (entregado ?p)))
)
```

Se ha creado un objeto por cada cocina, almacen y dormitorio. También hemos creado un objeto para el robot cocinero, los dos robots asistentes y para cada cliente. Se han creado cuatro slots que representan los dos contenedores que cada uno de los robots asistentes usa para llevar los productos. Se ha creado un objeto por cada tipo de producto o preparación mencionada en el enunciado, y se ha creado un identificador para cada uno de los pedidos, ya que una persona puede realizar más de un pedido.

El estado inicial es una fotografía de la configuración que aparece en el enunciado, indicando con

el predicado **esta\_en** donde están los robots, los productos y los clientes. Las preparaciones no están en ningún sitio en el estado inicial, ya que aún deben ser preparadas. Con el predicado **tiene** indicamos que slots corresponden a cada robot asistente. Y añadimos los pedidos que aparecen en la tabla que nos dan como ejemplo.

El estado objetivo usa el predicado **servido** (el plan acaba cuando todos los clientes han sido servidos por los robots disponibles). Se podría haber descrito listando los diez predicados **servido** (uno por pedido), pero en este caso se ha optado por la expresión **forall** en adl, ya que nos permite cambiar el número de pedidos del problema sin tener que cambiar la descripción del estado objetivo, y no nos añade complejidad (en FastForward el **forall** en los objetivos se instancia una sola vez generando los predicados individuales, con un coste lineal respecto al número de objetos a explorar).

Las notas se publicarán el día **25 de junio**.